

GROCERY INVENTORY

Frontend and full-stack project explained

g

grocerly.

A code-grounded walkthrough of the

What this project contains

Browser application: Vite serves a single HTML entry point.

UI implementation: vanilla JavaScript builds HTML strings and binds events.

Backend: Flask exposes JSON REST endpoints under /api.

Storage: Python in-memory lists and dictionaries for development.

Users: bearer-token login with admin and staff permissions.

The central idea

The frontend is a small single-page application. It

Browser <->

How the folders fit together

HTML shell

Application

Visual

REST API

Reference

What happens when the frontend opens

Creates `<div id="root">` and loads

Imports `styles.css`, reads

Shows login if no user exists;

Requests summary, products,

Connects buttons, forms,

`function`

The state object is the UI memory

```
const state = {  
  user:  
    readUser(),  
  page:  
    'Overview',  
  summary: null,  
  products: [],  
  suppliers: [],
```

user: who is signed in
page: current section
arrays: records loaded from Flask
modal: which dialog is open
billing: selected products and quantities
loading: whether requests are in progress

How screens are produced

renderLogin() creates the sign-in form.
renderApp() creates sidebar, header, page content, and optional modal.
renderPage() chooses a page-specific renderer.
renderOverview(), renderDirectory(), renderBilling(), renderReports(), and renderUsers() return HTML.
escapeHtml() protects values inserted into HTML tables and labels.

state.pagenderPage()

Overview |

state.page

Login and token flow

Login POST Flask Token localStorage

```
localStorage.setItem('grocery_token',
```

apiFetch() reads the token and adds
Authorization: Bearer <token>.

A 401 response calls logout().

Passwords are never stored in the frontend. Flask
stores password hashes.

Admin and staff roles

Administrator

Overview,
Inventory,
Purchases,

Staff member

Inventory and Billing
Can create sales
and purchases, but

```
@app.post('/api/users')
```


How the frontend talks to Flask

```
function  
apiFetch(path,  
options = {}) {  
  const token =  
    localStorage.getItem('processyToken')
```

```
  apiJson(path,  
    1. Calls  
    apiFetch() 2.  
    processyToken )  
  Checks
```

API base URL:

Dashboard loading and refresh behavior

`loadData()Promise.allSettled()` state `render()`

```
const calls = [  
  apiJson( '/summary' ),
```

loading-bar appears while requests are active
Promise.allSettled prevents one failed call from
stopping every view
successful records replace old state before the
next render

Why adding a user works only for admin

```
const routes =
```

Inventory, purchases, billing, and reports

Inventory	Purchase	Billing	Reports
GET /products	POST /purchases	POST /sales	GET /summary

Products are the shared record: purchases

What Flask is responsible for

Authentication: validates credentials and creates active bearer tokens.

Authorization: `require_auth()` checks token and allowed role.

Validation: required fields, duplicate usernames, product and supplier existence.

Business rules: stock cannot go below zero; at least one admin must remain.

Responses: JSON data plus meaningful HTTP status codes such as 201, 400, 401, 403, 404, and 409.

`/auth/login`

`/auth/me`

`/users`

`/summary`

`/products`

How the visual interface is organized

CSS variables define ink, muted text, borders, green, yellow, red, and blue tones.

The app shell uses a fixed sidebar and scrollable main content. Reusable visual classes cover panels, buttons, tables, metrics, alerts, forms, and modals.

DM Sans is used for body text; Manrope is used for headings and totals.

Media queries at 1050px and 700px adapt navigation, grids, forms, and reports.

Responsive

Desktop Sidebar
visible

Multi-column
metrics Mobile

How to start the application locally

# Terminal 1 -	# Terminal 2 -
backend cd	frontend cd
backend python -m	frontend npm
venv .venv	
.venv\Scripts\Activate.ps1	

A complete demonstration sequence

Important development notes

Storage is in memory: restarting Flask removes products, suppliers, purchases, sales, and newly created users.
database/seed_data.json is reference data only and is not loaded at startup.

The CSV download is minimal: it serializes object values and does not add robust quoting or column headers.

The API URL is hard-coded to localhost:5000, so deployment needs environment-based configuration.

The README account table is out of sync with the actual code: code uses liki and aliya as staff accounts.

Recommended

1. Add MySQL repository layer
2. Move API URL to environment

The complete mental model

Browser

state +

Flask

auth +

Records

in-memory

UI action -> API request -> validated

That loop explains nearly every